



NOLTE KÜCHEN · ERP

Guide développeur et dossier de passation — ERP Nolte Küchen

Tout ce qu'un développeur doit savoir pour comprendre, exploiter, modifier et faire évoluer l'ERP Nolte Küchen : architecture, base de données, sécurité, déploiement, sauvegardes, supervision, recettes de modification et dette technique connue.

14/09/2026

Sommaire

1. L'essentiel en une page	4
2. Architecture	6
2.1 Vue d'ensemble	6
2.2 Les services Docker	6
2.3 Le frontend	8
2.4 Le parcours d'une requête	8
3. Le dépôt	10
3.1 Arborescence	10
3.2 Branches	11
4. La base de données	12
4.1 Les tables par domaine	12
4.2 Le moteur de stock	13
4.3 Le circuit de vente en base	14
4.4 Projets	15
4.5 Fichiers (Storage)	15
5. Sécurité et permissions	16
5.1 Le modèle	16
5.2 L'évaluation d'une permission	16
5.3 Où les droits sont appliqués	16
5.4 Durcissement de l'hôte (état vérifié le 14/09/2026)	18
6. Développer en local	19
6.1 Prérequis	19
6.2 Démarrer la pile complète	19
6.3 Itérer sur le frontend	19
6.4 Travailler sur une copie des données de prod	20
7. Déployer et exploiter	21
7.1 Raccourci	21
7.2 Déployer une nouvelle version (manuel)	21
7.3 Déploiement continu (GitHub Actions)	21
7.4 Migrations de base de données	22
7.5 nginx et certificats TLS	23
7.6 Sauvegardes et restauration	23
7.7 Supervision et alertes	24
7.8 Retour arrière	24
7.9 Les scripts	25
7.10 Reprise après sinistre (serveur perdu)	25

8. Modifier l'application : recettes	26
8.1 Ajouter une page	26
8.2 Lire et écrire des données	26
8.3 Ajouter une permission ou un module	27
8.4 Écrire une fonction SQL (RPC)	27
8.5 Modifier un PDF	27
8.6 Réactiver l'arabe dans l'interface	27
8.7 Server function avec droits élevés	28
8.8 Mettre à jour la documentation	28
9. Mettre à niveau	29
10. Dette technique et problèmes connus	30
11. Checklist de passation	32
11.1 Accès à transmettre (hors de ce document, par canal sûr)	32
11.2 Première semaine du nouveau développeur	32
11.3 Contrôles réguliers	33
Annexe A. Variables d'environnement (.env)	34
Annexe B. Commandes utiles	35

1. L'essentiel en une page

Ce qu'est l'application. Un ERP interne pour un cuisiniste (Nolte Küchen, Maroc). Il couvre le catalogue produits, le stock multi-dépôts, le circuit de vente (devis → commande → vente → livraison → paiements), la facturation, les achats fournisseurs, les retours, les projets de pose de cuisine, les rendez-vous, la GED (documents) et les rapports. Il est utilisé en français, les montants sont en dirhams (DH) TTC.

Comment il est construit.

- **Frontend + SSR** : TanStack Start (React 19, TanStack Router et Query, Tailwind CSS 4, composants shadcn/Radix), compilé par Vite et servi par un petit serveur Node 22.
- **Backend** : une pile **Supabase auto-hébergée** : PostgreSQL 15, GoTrue (authentification), PostgREST (API REST générée depuis le schéma), Storage API sur MinIO (fichiers), le tout derrière la passerelle Kong.
- **L'essentiel de la logique métier est en base** : fonctions PL/pgSQL (RPC), triggers pour le stock, politiques RLS pour les droits.
- **Tout tourne dans Docker Compose** derrière nginx (TLS Let's Encrypt), sur un seul VPS.

Origine. Le projet a démarré sur la plateforme **Lovable** (Lovable Cloud = Supabase hébergé). Il a ensuite été migré en auto-hébergement, voir `MIGRATION.md`. Des traces de Lovable subsistent (commentaires générés, `wrangler.jsonc`, config Vite Cloudflare remplacée au build).

Où il tourne.

Élément	Valeur
Serveur	VPS OVH, Ubuntu 26.04 LTS, 2 vCPU / 3,8 Go RAM / 38 Go disque, sans swap
Accès	SSH par clé uniquement (pas de root, pas de mot de passe), utilisateur <code>farah</code> (groupes <code>sudo</code> , <code>docker</code>)
Répertoire	<code>/opt/nolte</code> (clone Git du dépôt)
Application	<code>https://noltenador.digital</code>
API Supabase	<code>https://supabase.noltenador.digital</code>
Supervision	<code>https://noltenador.digital/_ops/</code> (Grafana) et <code>/_prom/</code> (Prometheus)
Dépôt Git	<code>github.com/Ahmed61911/noltekuchen</code> , branche de production <code>main</code>

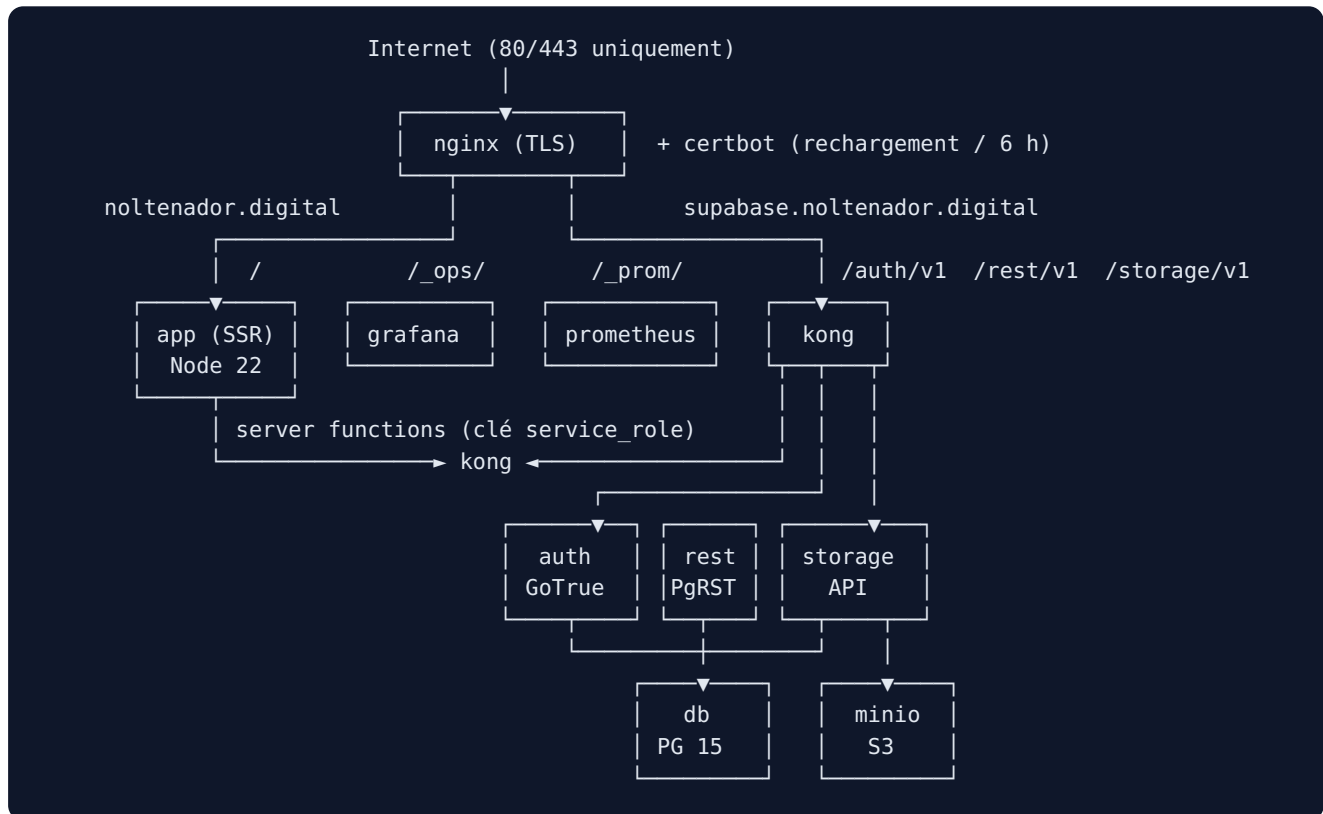
Les trois commandes à connaître.

```
cd /opt/nolte
C="docker compose -f docker-compose.yml -f docker-compose.prod.yml -f docker-
compose.monitoring.yml"
$C ps                # état de tous les services
$C logs -f --tail=100 app # journaux d'un service
./scripts/verify-backup.sh # prouve que la dernière sauvegarde est restaurable
```

Important : sur le serveur, lancez **toujours** `docker compose` avec au minimum `-f docker-compose.yml -f docker-compose.prod.yml` . Le fichier de base seul **republie les ports de Postgres (5432), Kong (8000), MinIO (9001) et de l'app (8080) sur Internet**, en contournant le pare-feu (Docker écrit ses propres règles iptables). C'est arrivé le 08/09/2026.

2. Architecture

2.1 Vue d'ensemble



Le navigateur parle directement à l'API Supabase. La quasi-totalité des lectures et écritures passe par `supabase-js` côté client : `supabase.from(...)` pour PostgREST et `supabase.rpc(...)` pour les fonctions SQL, avec le JWT de l'utilisateur. **La sécurité repose donc entièrement sur les politiques RLS de Postgres**, et non sur le code React. Masquer un bouton dans l'interface n'est jamais une protection.

Seules les opérations d'administration des comptes (création d'utilisateur, réinitialisation de mot de passe, rôles) passent par des **server functions** TanStack exécutées dans le conteneur `app`. Elles utilisent la clé `service_role`, qui contourne RLS, après avoir vérifié que l'appelant est administrateur.

2.2 Les services Docker

Service	Image	Rôle	Fichier de config
db	supabase/ postgres:15.6.1.146	Base de données (volume <code>pg_data</code>)	backend/volumes/db/init/ 00-roles.sh
auth	supabase/gotrue:v2.158.1	Comptes, connexion, JWT, e-mails	variables <code>GOTRUE_*</code> , <code>SMTP_*</code>

Service	Image	Rôle	Fichier de config
rest	postgres/ postgres:v12.2.3	API REST automatique sur le schéma <code>public</code>	—
storage	supabase/storage- api:v1.11.13	Fichiers (buckets, RLS sur <code>storage.objects</code>)	—
minio	minio/ minio:RELEASE.2024-08-17...	Stockage objet S3 (volume <code>minio_data</code>)	—
minio-init	minio/mc	Tâche ponctuelle : crée les buckets	<code>docker-compose.yml</code>
kong	kong:2.8	Passerelle : <code>/auth/</code> <code>v1</code> , <code>/rest/v1</code> , <code>/</code> <code>storage/v1</code>	<code>backend/volumes/api/</code> <code>kong.yml</code>
db-migrate	supabase/postgres	Tâche ponctuelle : applique <code>database/</code> <code>migrations</code> au premier démarrage	<code>backend/volumes/db/run-</code> <code>app-migrations.sh</code>
app	build frontend/Dockerfile	Application TanStack Start (SSR + assets), limite 1 Go	<code>frontend/</code>
nginx	nginx:1.27-alpine	Reverse proxy TLS, rate limiting, auth basic <code>/_ops</code>	<code>nginx/</code>
certbot	certbot/certbot:v2.11.0	Renouvellement Let's Encrypt toutes les 12 h	<code>docker-compose.prod.yml</code>
smtp	mailhog	Dev uniquement (désactivé en prod)	—
prometheus , grafana , node-exporter , postgres- exporter , blackbox- exporter	voir <code>docker-</code> <code>compose.monitoring.yml</code>	Supervision (chapitre 8)	<code>monitoring/</code>
loki , promtail	profil <code>logs</code>	Agrégation de logs, non démarrée (mémoire insuffisante)	<code>monitoring/</code>

Les trois fichiers Compose :

- `docker-compose.yml` : base, utilisable seule **en développement local uniquement**.

- `docker-compose.prod.yml` : surcouche de prod (`restart: always` , suppression des ports avec `!reset []` , nginx, certbot, rotation des logs, limites mémoire).
- `docker-compose.monitoring.yml` : supervision (tous les services ont un `mem_limit`).

2.3 Le frontend

Sujet	Détail
Framework	TanStack Start ^1.168 (SSR), TanStack Router (routage par fichiers), React 19
Données	@tanstack/react-query (cache, invalidations), @supabase/supabase-js
UI	Tailwind CSS 4, composants shadcn (Radix) dans <code>src/components/ui</code> , briques maison dans <code>src/components/data</code>
Graphiques	recharts
PDF	jspdf + jspdf-autotable , générés dans le navigateur (<code>src/lib/*-pdf.ts</code>)
Excel	xlsx (SheetJS, installé depuis le CDN SheetJS)
Notifications	sonner (toasts)
Gestionnaire de paquets	Bun (<code>bun.lock</code>) ; le Dockerfile l'installe via npm

Build Node. La config Vite d'origine (Lovable) cible Cloudflare Workers. Le `frontend/Dockerfile` la remplace au build :

- `frontend/vite.config.node.ts` → `vite.config.ts` ;
- `frontend/server-node.ts` → `src/server.ts` ;
- `frontend/serve.mjs` est un serveur HTTP Node minimal : il sert `dist/client/` et transmet le reste au handler `fetch` produit par TanStack Start.

Node **22** est obligatoire : `supabase-js` a besoin du `WebSocket` natif, absent de Node 20.

Variables `VITE_*` . Elles sont **injectées au moment du build** (arguments Docker). Changer `VITE_SUPABASE_URL` ou la clé publique impose de **reconstruire** l'image `app` , un simple redémarrage ne suffit pas.

2.4 Le parcours d'une requête

1. Le navigateur charge `https://noltenador.digital/commandes...` : nginx → `app:3000` (SSR, puis hydratation React).
2. `AuthProvider` (`src/lib/auth.tsx`) récupère la session GoTrue, stockée dans le navigateur, puis charge les rôles depuis `user_roles` .

3. Une page lit les données : `supabase.from("orders").select(...)` → `https://supabase.noltenador.digital/rest/v1/orders` → nginx (rate limit `api_zone`) → Kong (vérifie la clé `apikey`) → PostgREST → Postgres, **en tant que rôle `authenticated`, avec le JWT** → les politiques RLS filtrent les lignes.
4. Une action métier appelle une RPC : `supabase.rpc("validate_order", ...)` → même chemin → la fonction SQL s'exécute dans une transaction ; les triggers ajustent le stock.
5. Un fichier : `supabase.storage.from("product-images").upload(...)` → Storage API → MinIO, avec RLS sur `storage.objects`.

3. Le dépôt

3.1 Arborescence

Chemin	Contenu
<code>src/routes/</code>	Pages (routage par fichiers). <code>_app.*.tsx</code> = pages protégées dans la mise en page avec menu. <code>\$id</code> = paramètre.
<code>src/routeTree.gen.ts</code>	Généré automatiquement par le plugin TanStack Router : ne pas éditer.
<code>src/components/ui/</code>	Composants shadcn (bas niveau).
<code>src/components/data/</code>	Briques de pages de données : <code>PageHeader</code> , <code>Toolbar</code> , <code>StatCard</code> , <code>StatusBadge</code> , <code>EmptyState</code> , <code>ErrorState</code> , pagination, squelettes.
<code>src/components/app-sidebar.tsx</code>	Menu latéral et filtrage par permission.
<code>src/components/app-header.tsx</code>	Barre supérieure et notifications.
<code>src/lib/auth.tsx</code>	Session, rôles, <code>isAdmin</code> .
<code>src/lib/permissions.tsx</code>	Hook <code>usePermissions()</code> → <code>can(module, action)</code> .
<code>src/lib/users.functions.ts</code> , <code>roles.functions.ts</code>	Server functions d'administration (clé <code>service_role</code>).
<code>src/lib/*-pdf.ts</code>	Mise en page des PDF (devis, commande, facture), logos.
<code>src/lib/money.ts</code>	Calculs de lignes et totaux (<code>computeLine</code> , <code>computeTotals</code> , <code>round2</code>).
<code>src/lib/stock-by-warehouse.ts</code>	Stock par dépôt, calculé depuis les mouvements.
<code>src/lib/audit-log.ts</code>	<code>logAction()</code> → table <code>audit_logs</code> .
<code>src/lib/i18n.tsx</code>	Dictionnaires FR/AR (AR désactivé , voir 10).
<code>src/integrations/supabase/</code>	Clients Supabase (navigateur, serveur), middleware d'authentification, types générés (<code>types.ts</code>).
<code>database/migrations/</code>	Migrations SQL de l'application (source de vérité du schéma).
<code>supabase/migrations/</code>	Anciennes migrations de l'époque Lovable (historique ; ne sont plus appliquées).
<code>backend/volumes/</code>	Config Kong et scripts d'initialisation Postgres.
<code>frontend/</code>	Dockerfile et adaptation Node.
<code>nginx/</code>	<code>nginx.conf</code> , <code>conf.d/app.conf.template</code> (rendu → <code>app.conf</code> , non versionné), <code>ops.htpasswd</code> (non versionné).
<code>monitoring/</code>	Prometheus, règles et tableaux de bord Grafana, blackbox, promtail.
<code>scripts/</code>	Exploitation (voir 7.9).

Chemin	Contenu
<code>docs/</code>	Ces guides, les outils de génération PDF, notes de design.
<code>agent_docs/</code>	Notes de travail d'assistants IA précédents (historique, non normatif).
<code>DEPLOY.md</code> , <code>MIGRATION.md</code>	Runbooks d'origine (en anglais). Attention : <code>DEPLOY.md</code> parle d'un serveur Hetzner, alors que la production actuelle est chez OVH.
<code>.github/workflows/deploy.yml</code>	Déploiement continu sur push <code>main</code> (voir 7.3).

3.2 Branches

Branche	Rôle
<code>main</code>	Production. Tout push déclenche le workflow de déploiement.
<code>ops/image-perms-movement-date-monitoring</code>	Branche de travail actuellement extraite sur le serveur, identique à <code>main</code> au 14/09/2026.
<code>v1.1a-redesign</code> , <code>v1.1b</code>	Anciennes branches de fonctionnalités (historique).

4. La base de données

4.1 Les tables par domaine

Domaine	Tables
Comptes et droits	<code>profiles</code> (1-1 avec <code>auth.users</code>), <code>user_roles</code> , <code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>user_permissions</code>
Catalogue	<code>products</code> , <code>categories</code> , <code>suppliers</code> , <code>warehouses</code>
Stock	<code>stock_movements</code> (journal immuable ; les quantités de <code>products</code> en découlent)
Clients	<code>customers</code>
Vente	<code>quotes</code> + <code>quote_items</code> , <code>orders</code> + <code>order_items</code> + <code>order_payments</code> , <code>sales</code> + <code>sale_items</code> + <code>sale_payments</code> , <code>invoices</code> + <code>invoice_items</code>
Achats et retours	<code>purchase_orders</code> + <code>purchase_order_items</code> , <code>returns</code> + <code>return_items</code>
Projets	<code>projects</code> , <code>project_stages</code> , <code>project_attachments</code> , <code>project_activity</code>
Divers	<code>appointments</code> , <code>documents</code> , <code>document_history</code> , <code>audit_logs</code>
Technique	<code>_schema_migrations</code> (migrations appliquées)

Pour le détail des colonnes : `docker exec nolte-db-1 psql -U postgres -d postgres -c '\d public.orders'`, ou le fichier généré `src/integrations/supabase/types.ts`.

Numérotation des documents (fonctions `generate_*_number`, format préfixe + année-mois + séquence) :

Document	Préfixe
Devis	DEV -
Commande	CMD -
Vente	VTE -
Facture	FAC -
Achat	ACH -
Retour client (avoir)	AV -
Retour fournisseur	RF -

4.2 Le moteur de stock

C'est la partie la plus sensible. Règle d'or : **on ne modifie jamais `products.stock_quantity` directement**. Toute variation passe par une ligne dans `stock_movements`.

- **`apply_stock_movement()`** (trigger `BEFORE INSERT` sur `stock_movements`, `SECURITY DEFINER`) :
 - verrouille la ligne produit (`FOR UPDATE`) ;
 - calcule le sens selon `type` : `in`, `purchase`, `customer_return` et `inventory` ajoutent ; `out`, `sale` et `supplier_return` retirent ; `damaged` fait passer du stock vendable au stock endommagé ;
 - **refuse de passer sous zéro** (erreur 23514 « Stock insuffisant ») ;
 - renseigne `stock_before` / `stock_after` ;
 - met à jour `products.stock_quantity` ou `damaged_quantity`. Le drapeau `to_damaged` fait porter un mouvement sur le stock endommagé.
- **`move_document_stock(...)`** (`SECURITY DEFINER`) : helper générique qui crée les mouvements de toutes les lignes d'un document (commande, vente, facture, achat) dans un sens donné.
- **Triggers de documents** (tous `SECURITY DEFINER`) :

Trigger	Événement	Effet
<code>trg_orders_stock</code> → <code>apply_order_stock()</code>	<code>UPDATE</code> de <code>orders.status</code>	Passage à <code>delivered</code> → sortie ; sortie de <code>delivered</code> (annulation) → réintégration
<code>trg_orders_stock_delete</code> → <code>revert_order_stock_on_delete()</code>	<code>DELETE</code>	Réintègre si la commande était livrée
<code>trg_sale_items_stock</code> → <code>apply_sale_stock_deferred()</code>	<code>INSERT</code> de ligne de vente	Vente sans commande (comptoir) : sortie ; vente liée à une commande : rien (le stock sort à la livraison)
<code>trg_sales_stock_delete</code> → <code>revert_sale_stock_on_delete()</code>	<code>DELETE</code>	Réintègre
<code>trg_invoice_stock</code> → <code>apply_invoice_stock()</code>	<code>UPDATE</code> de <code>invoices.status</code>	Entrée dans <code>pending</code> / <code>paid</code> → sortie ; sortie de ces statuts → réintégration
<code>trg_invoice_stock_delete</code>	<code>DELETE</code>	Réintègre
<code>trg_apply_purchase_order_stock</code>	<code>UPDATE</code> de <code>purchase_orders.status</code>	Passage à <code>received</code> → entrée au coût des lignes ; annulation → sortie
<code>trg_purchase_orders_stock_delete</code>	<code>DELETE</code>	Retire si l'achat était reçu

Des colonnes drapeaux (`stock_applied` , etc.) évitent d'appliquer deux fois le même document.

- **Stock par dépôt** : il n'y a pas de table stock × dépôt. Le front calcule la répartition à partir des mouvements (`src/lib/stock-by-warehouse.ts` , hook `useStockByWarehouse().depotsFor(productId)`). Le « dépôt d'un produit » découle donc de son stock (commit `643db54`), et non de `products.warehouse_id` .

Important : une facture en `pending` / `paid` **sort** le stock, **indépendamment** de la commande ou de la vente. Facturer avec des lignes produits une commande déjà livrée sort le stock deux fois. C'est un piège fonctionnel connu (voir 10).

4.3 Le circuit de vente en base

Étape	Appel	Détail
Créer un devis	<code>insert quotes + quote_items</code>	Totaux recalculés côté client (<code>recomputeTotals</code>)
Accepter	<code>rpc accept_quote(_quote_id)</code>	Crée la commande <code>pending</code> et ses lignes ; idempotent (<code>already: true</code>)
Créer une commande	<code>rpc create_order(_order jsonb, _items jsonb)</code>	Transactionnel
Valider	<code>rpc validate_order(_order_id)</code>	Crée la vente liée (<code>stock_applied=false</code>), copie les lignes et reprend les paiements déjà saisis ; idempotent
Livrer	<code>update orders set status='delivered'</code>	Le trigger sort le stock (<code>deliver_order()</code> existe aussi)
Annuler	<code>rpc cancel_order / cancel_sale</code>	Annule le couple commande/vente ; le trigger réintègre si livré
Vente comptoir	<code>rpc create_sale_with_order</code>	Crée la vente et une commande <code>validated</code>
Facture	<code>rpc create_invoice</code>	Le statut initial peut déjà sortir le stock
Achat	<code>rpc create_purchase_order ; puis update status='received'</code>	
Retour	<code>rpc create_return / cancel_return</code>	Crée ou inverse les mouvements
Paielements	<code>insert order_payments / sale_payments</code>	Triggers <code>sync_order_payment_status / sync_sale_payment_status</code> : <code>paid_amount</code> et <code>payment_status</code>

Les RPC `create_*` , `validate_order` , `cancel_*` et `accept_quote` sont **SECURITY INVOKER** : elles s'exécutent avec les droits de l'utilisateur, donc **sous RLS**. Valider une commande exige par exemple `orders.update` **et** `sales.create` .

4.4 Projets

- `trg_seed_project_stages` (AFTER INSERT sur `projects`) crée les 9 étapes : `design` , `client_validation` , `supplier_order` , `goods_reception` , `preparation` , `delivery` , `installation` , `quality_check` , `completed` .
- `trg_recompute_project_progress` recalcule l'avancement à chaque modification d'étape.

4.5 Fichiers (Storage)

Bucket	Public	Usage	Politiques
<code>product-images</code>	non	Photos produits	Lecture : authentifiés ; écriture : <code>user_has_permission(uid, 'products', 'update')</code> (migration <code>20260908120000</code>)
<code>documents</code>	non	GED et pièces jointes de projets	Selon propriétaire / admin

Le bucket S3 interne de Storage API s'appelle `STORAGE_S3_BUCKET` (`supabase-storage`) dans MinIO. Les objets y sont rangés sous `supabase-storage/self-hosted/<bucket>/...` .

5. Sécurité et permissions

5.1 Le modèle

```
auth.users —1:1— profiles
├── user_roles (user_id, role app_role, role_key text)
│   └── roles (key, label, is_system)
│       └── role_permissions (role, permission_id)
│           └── permissions (module, action, label)
└── user_permissions (user_id, permission_id, granted) ← surcharges individuelles
```

- **app_role** (enum): admin, employee, manager, commercial, warehouse, accountant. Les rôles créés depuis l'écran *Rôles* utilisent **role_key** (texte); COALESCE(role_key, role::text) fait le lien.
- **Modules**: customers, orders, products, projects, purchase_orders, quotes, reports, sales, stock, suppliers, users.
- **Actions**: view, create, update, delete, export, print, plus des actions spécifiques: orders.validate, orders.cancel, sales.invoice, stock.in, stock.out, stock.adjust, reports.export_excel, reports.export_pdf, users.manage_permissions.
- **Nouvel inscrit**: le trigger handle_new_user() crée le profil et le rôle employee. En pratique les comptes sont créés par un admin (inscription publique désactivée: GOTRUE_DISABLE_SIGNUP).

5.2 L'évaluation d'une permission

user_has_permission(_user_id, _module, _action) (SECURITY DEFINER , STABLE) applique, dans l'ordre :

1. une **surcharge individuelle** (user_permissions.granted) si elle existe, **même false**, et **même pour un admin** ;
2. sinon, **admin** → true ;
3. sinon, une permission accordée via le rôle (role_permissions);
4. sinon false .

has_role(uid, role) teste simplement l'appartenance à un rôle. get_user_permissions(uid) renvoie la matrice complète au front.

5.3 Où les droits sont appliqués

Couche	Mécanisme	Portée réelle
Postgres RLS	user_has_permission(auth.uid(), '<module>', '<action>') ou has_role(..., 'admin')	La seule barrière de sécurité effective.

Couche	Mécanisme	Portée réelle
Server functions	<code>assertAdmin()</code> puis client <code>service_role</code>	Administration des comptes et des rôles
Menu	<code>can(module, 'view')</code> dans <code>app-sidebar.tsx</code>	Confort (masquage)
Pages	<code>isAdmin</code> ou <code>can()</code> pour certains boutons	Confort (masquage)

État des politiques par table (au 14/09/2026) :

Tables
<code>customers</code> , <code>orders</code> , <code>quotes</code> , <code>sales</code> , <code>invoices</code> , <code>projects</code> , <code>purchase_orders</code> , <code>suppliers</code>
Lignes et paiements (<code>order_items</code> , <code>order_payments</code> , <code>sale_items</code> , <code>sale_payments</code> , <code>purchase_order_items</code> , <code>project_stages</code> , <code>project_attachments</code>)
<code>invoice_items</code>
<code>products</code>
<code>warehouses</code>
<code>stock_movements</code>
<code>appointments</code> , <code>documents</code>
<code>audit_logs</code>
<code>profiles</code>

Tables`user_roles , roles , permissions , role_permissions``returns , return_items`

5.4 Durcissement de l'hôte (état vérifié le 14/09/2026)

- **UFW** actif : entrées refusées par défaut, seuls 22, 80 et 443 autorisés (IPv4 et IPv6).
- Seul **nginx** publie des ports (80/443). `db` , `kong` , `minio` et `app` utilisent `ports: !reset []` .
- **SSH** : pas de connexion root, pas de mot de passe. **fail2ban** actif (jails `sshd` et `nginx-ratelimit` sur les 429). **unattended-upgrades** actif.
- `nginx` : limitation de débit `auth_zone` sur `/auth/v1/(token|signup|recover|otp|verify|magiclink)` et `api_zone` sur le reste de l'API ; en-têtes de sécurité.
- `/_ops/` et `/_prom/` : authentification basique `nginx` (`nginx/conf.d/ops.htpasswd` , utilisateur `ops`), plus la connexion Grafana.
- Prometheus : `--web.enable-lifecycle` **retiré** (pas d'arrêt à distance).
- `postgres-exporter` : rôle dédié en lecture seule `nolte_exporter` (`pg_monitor`), et non le superutilisateur.
- Une entrée sudoers limitée `/etc/sudoers.d/90-claude-ufw` (`ufw` uniquement) a été ajoutée pendant l'audit : **à supprimer** si elle n'est plus utile.

6. Développer en local

6.1 Prérequis

Docker (Desktop ou Engine + plugin Compose), `openssl`, `python3`, `bash`, et Bun ou Node 22 pour travailler hors conteneur.

6.2 Démarrer la pile complète

```
git clone git@github.com:Ahmed61911/noltekuchen.git
cd noltekuchen
scripts/bootstrap.sh      # génère .env : secrets aléatoires + clés anon/service_role signées
docker compose up -d      # premier démarrage ~2 min ; db-migrate applique database/migrations
scripts/create-admin.sh admin@example.com 'MotDePasse!123'
```

Service	URL locale
Application	http://localhost:8080
API Supabase (Kong)	http://localhost:8000
Console MinIO	http://localhost:9001
MailHog (e-mails capturés)	http://localhost:8025
Postgres	localhost:5432

Réinitialiser complètement : `docker compose down -v`, puis `up -d`.

6.3 Itérer sur le frontend

Le conteneur `app` est une image de production : chaque changement de code impose `docker compose build app && docker compose up -d app`. Pour itérer vite, lancez Vite en local contre l'API Docker :

```
bun install
# dans .env : VITE_SUPABASE_URL=http://localhost:8000 et la clé publique générée
bun run dev
```

Bon à savoir : la config Vite du dépôt est celle de Lovable ([@lovable.dev/vite-tanstack-config](https://lovable.dev/vite-tanstack-config), cible Cloudflare). Si `bun run dev` pose problème, utilisez la config Node : `bunx vite dev --config frontend/vite.config.node.ts`. Les server functions ont besoin de `SUPABASE_URL` et `SUPABASE_SERVICE_ROLE_KEY` dans l'environnement.

Qualité : `bun run lint` (ESLint) et `bun run format` (Prettier). Il n'y a **pas de tests automatisés** dans le dépôt.

6.4 Travailler sur une copie des données de prod

Pour reproduire un bug sur des données réelles sans toucher la prod :

1. Récupérez une archive `backups/backup-*.tar.gz` du serveur (`scp`).
2. En local, pile démarrée : `scripts/restore.sh backups/backup-XXXX.tar.gz` .

Important : ces archives contiennent des données personnelles (clients, comptes, hachages de mots de passe). Ne les stockez pas hors d'un disque chiffré et supprimez-les après usage.

7. Déployer et exploiter

7.1 Raccourci

Toutes les commandes ci-dessous supposent :

```
cd /opt/nolte
C="docker compose -f docker-compose.yml -f docker-compose.prod.yml -f docker-
compose.monitoring.yml"
```

7.2 Déployer une nouvelle version (manuel)

```
cd /opt/nolte
git fetch origin && git status          # vérifier : aucune modification locale non commitée
git checkout main && git pull --ff-only
$C build app                          # ~3-5 min ; surveiller la RAM (pas de swap)
./scripts/migrate.sh                  # applique les nouvelles migrations + NOTIFY pgrst
$C up -d app                          # coupure de quelques secondes

# Seulement si nginx/ a changé :
./scripts/render-nginx.sh && docker exec nolte-nginx-1 nginx -t && docker exec nolte-nginx-1
nginx -s reload
# Seulement si kong.yml a changé :
$C up -d --force-recreate kong
# Seulement si docker-compose.monitoring.yml ou monitoring/ a changé :
$C up -d --no-deps prometheus grafana postgres-exporter
```

Vérification : `curl -sI https://noltenador.digital/ | head -1` (200 ou 307), puis un parcours rapide dans le navigateur.

7.3 Déploiement continu (GitHub Actions)

`.github/workflows/deploy.yml` se déclenche à chaque push sur `main` :

1. connexion SSH au serveur (secrets `SSH_HOST`, `SSH_USER`, `SSH_KEY`, `SSH_PORT`, `DEPLOY_PATH`) ;
2. `git fetch` puis `git checkout main && git reset --hard origin/main` ;
3. `build app` → `scripts/migrate.sh` → `up -d app` ;
4. si `nginx/` a changé : rendu et **restart** de nginx ;
5. `docker image prune -f`.

Important :

- `reset --hard` **écrase toute modification locale non commitée sur le serveur.** Ne modifiez jamais de fichier suivi directement sur le serveur sans le commiter aussitôt.
- Le workflow **ne redéploie ni la supervision, ni Kong, ni certbot.**
- Vérifiez dans *GitHub* → *Settings* → *Secrets and variables* → *Actions* que les secrets existent. Leur présence n'a pas été vérifiée lors de la passation. Sans eux, les pushes ne déploient rien et il faut suivre 7.2.
- Une clé de déploiement GitHub en écriture (`nolte-prod-deploy` , `~/.ssh/nolte_github` sur le serveur) permet de pousser depuis le serveur : `GIT_SSH_COMMAND="ssh -i ~/.ssh/nolte_github -o IdentitiesOnly=yes" git push git@github.com:Ahmed61911/noltekuchen.git HEAD:main` .

7.4 Migrations de base de données

- Emplacement : `database/migrations/AAAAMMJJHHMMSS_description.sql` , appliquées **par ordre alphabétique**.
- Suivi : `public._schema_migrations(filename)` . Un fichier déjà appliqué **n'est jamais rejoué** : ne modifiez jamais une migration publiée, créez-en une nouvelle.
- Application : `scripts/migrate.sh` (via `docker compose exec db psql`), qui envoie ensuite `NOTIFY pgrst, 'reload schema'` . **Sans ce NOTIFY, PostgREST ne voit pas les nouvelles colonnes ou fonctions** (erreur `PGRST202`).
- Il n'y a **pas de migrations descendantes**. Pour annuler : une nouvelle migration inverse, ou une restauration.
- `scripts/migrate.sh` n'utilise que `docker-compose.yml` pour `exec` : sans danger, car `exec` ne recrée aucun conteneur.

Écrire une bonne migration :

```
-- Pourquoi (le problème), quoi (la solution), comment c'est vérifié.
begin;

alter table public.orders add column if not exists delivery_address text;

-- Toujours penser RLS pour une nouvelle table :
-- alter table public.x enable row level security;
-- create policy x_select on public.x for select to authenticated
--   using (public.user_has_permission(auth.uid(), 'orders', 'view'));

commit;
```

Tester avant la prod : restaurez la dernière sauvegarde en local (6.4), appliquez la migration, et vérifiez l'application. Mettez ensuite à jour `src/integrations/supabase/types.ts` si des colonnes changent (générateur Supabase, ou édition manuelle cohérente).

7.5 nginx et certificats TLS

- `nginx/conf.d/app.conf.template` est rendu en `app.conf` par `scripts/render-nginx.sh` (substitution des domaines).
- `upstream` dynamiques (`resolver 127.0.0.11`) : nginx démarre même si un service est absent, et suit les changements d'IP des conteneurs.
- **Certificats** : `certbot` exécute `certbot renew` toutes les 12 h sur le volume `letsencrypt` , et **nginx se recharge toutes les 6 h** (boucle dans la commande du service) pour prendre en compte les certificats renouvelés. Vérifier l'expiration :

```
echo | openssl s_client -connect noltенador.digital:443 -servername noltенador.digital 2>/dev/null | openssl x509 -noout -enddate
```

- Premier certificat ou nouveau domaine : `scripts/init-letsencrypt.sh` (`--staging` d'abord).

7.6 Sauvegardes et restauration

Quoi	Quand	Où	Journal
<code>scripts/backup.sh</code> : <code>pg_dump</code> complet (<code>public</code> , <code>auth</code> , <code>storage</code>) + miroir des buckets MinIO → <code>backups/backup-AAAAMMJJ-HHMMSS.tar.gz</code> , 14 archives conservées	tous les jours à 03:00 (cron de <code>farah</code>)	<code>/opt/nolte/backups/</code>	<code>/opt/nolte/logs/nolte-backup.log</code>
<code>scripts/verify-backup.sh</code> : restaure la dernière archive dans un Postgres jetable sans réseau et compare utilisateurs, objets et tables avec la prod	le dimanche à 04:30	—	<code>/opt/nolte/logs/nolte-verify-backup.log</code>
<code>scripts/backup-offsite.sh</code> (restic)	non configuré	—	—

Important : toutes les sauvegardes sont sur le même serveur. Si le disque ou le VPS est perdu, tout est perdu. Configurer la copie hors site est la priorité n°1 de la passation (voir 11) : renseigner `RESTIC_REPOSITORY` et `RESTIC_PASSWORD` (plus les identifiants du fournisseur) dans `.env` , lancer `restic init` , puis relancer `scripts/install-cron.sh` .

Restaurer (destructif, demande confirmation) :

```
./scripts/restore.sh backups/backup-AAAAMMJJ-HHMMSS.tar.gz
```

Le script rejoue d'abord les `DROP` du dump en tolérant les erreurs, puis purge `auth` et `storage` (`CASCADE`), et recrée tout **dans une transaction unique** (`scripts/lib/restore-db.sh`). Il remet ensuite les propriétaires des schémas `auth` et `storage` , et réinjecte les fichiers MinIO. Ce fonctionnement a été testé sur base vide **et** sur base peuplée.

Tester une sauvegarde à la demande : `./scripts/verify-backup.sh` (sortie `PASS` / `FAIL`).

7.7 Supervision et alertes

Outil	Accès	Rôle
Grafana	https://noltenador.digital/_ops/ : auth basic <code>ops</code> , puis compte <code>admin</code> (<code>GRAFANA_ADMIN_PASSWORD</code> dans <code>.env</code>)	Tableau de bord « Nolte overview », alertes
Prometheus	https://noltenador.digital/_prom/ (auth basic <code>ops</code>)	Métriques brutes, <i>Status</i> → <i>Targets</i>

Cibles collectées (`monitoring/prometheus.yml`) : `prometheus` , `node` (hôte), `postgres` , `blackbox_app` (le site répond avec le bon contenu), `blackbox_api` (santé de l'API), `blackbox_tls` (expiration des certificats).

Règles d'alerte (`monitoring/grafana/provisioning/alerting/rules.yml`) :

- disque > 85 % ;
- mémoire disponible < 300 Mo ;
- application qui ne sert pas le contenu attendu ;
- santé de l'API Supabase en échec ;
- Postgres injoignable ;
- certificat TLS expirant sous 14 jours ;
- cible Prometheus tombée.

Envoi par e-mail : point de contact `nolte-email` → **noltenador@gmail.com** (`contact-points.yml`), via le SMTP de `.env` (Gmail et mot de passe d'application). **Activation en attente** au 14/09/2026 :

```
./scripts/set-gmail-smtp.sh noltenador@gmail.com # demande le mot de passe d'application sans
l'afficher
$C up -d --no-deps grafana auth
# puis Grafana → Alerting → Contact points → nolte-email → Test
```

Ce même SMTP fait fonctionner les e-mails de GoTrue (réinitialisation de mot de passe, invitations), aujourd'hui **non fonctionnels** car les valeurs SMTP sont des exemples.

7.8 Retour arrière

- **Code seul** : `git checkout <sha-sain>` → `$C build app` → `$C up -d app` . Remettez ensuite `main` au bon état, sinon le prochain push redéploiera la version fautive.
- **Migration fautive** : nouvelle migration corrective, ou restauration de la sauvegarde précédant le déploiement (perte des écritures intermédiaires).

7.9 Les scripts

Script	Usage
<code>bootstrap.sh</code>	Génère <code>.env</code> (secrets et clés JWT) pour une nouvelle installation
<code>create-admin.sh email 'mdp'</code>	Crée un compte administrateur
<code>migrate.sh</code>	Applique les migrations en attente
<code>backup.sh</code> / <code>restore.sh</code> / <code>verify-backup.sh</code>	Sauvegarde, restauration, preuve de restaurabilité
<code>lib/restore-db.sh</code>	Logique de rejeu d'un dump (partagée)
<code>backup-offsite.sh</code>	Envoi hors site (restic)
<code>install-cron.sh</code>	Installe les tâches cron (idempotent)
<code>render-nginx.sh</code>	Rend <code>app.conf</code> depuis le modèle
<code>init-letsencrypt.sh</code>	Premier certificat TLS
<code>create-exporter-role.sh</code>	(Re)crée le rôle <code>nolte_exporter</code>
<code>set-gmail-smtp.sh</code>	Configure le SMTP Gmail dans <code>.env</code>
<code>harden-host.sh</code>	Durcissement initial de l'hôte (UFW, fail2ban, démon Docker)
<code>export-lovable.sh</code> / <code>import-lovable.sh</code>	Migration unique depuis Lovable Cloud (historique)

7.10 Reprise après sinistre (serveur perdu)

1. Nouveau VPS : suivre `DEPLOY.md` §1 à §4 (utilisateur, Docker, clone, `harden-host.sh`, `bootstrap.sh`, DNS, `init-letsencrypt.sh`, `up -d`).
2. Récupérer une archive (hors site, voir 7.6) et `scripts/restore.sh <archive>`.
3. Repointer les DNS `noltenador.digital` et `supabase.noltenador.digital`.
4. Redémarrer la supervision : `.env` (`GRAFANA_ADMIN_PASSWORD`, `POSTGRES_EXPORTER_PASSWORD`, SMTP), `create-exporter-role.sh`, `ops.htpasswd`, puis `$C up -d`.
5. Réinstaller les crons : `scripts/install-cron.sh`.
6. Les utilisateurs se reconnectent : les sessions sont invalidées si le secret JWT change, mais les mots de passe sont conservés.

8. Modifier l'application : recettes

8.1 Ajouter une page

1. Créez `src/routes/_app.ma-page.tsx` :

```
import { createFileRoute } from "@tanstack/react-router";
import { PageHeader } from "@components/data/page-header";

export const Route = createFileRoute("/_app/ma-page")({ component: MaPage });

function MaPage() {
  return <PageHeader title="Ma page" subtitle="À quoi elle sert" />;
}
```

1. Le plugin régénère `src/routeTree.gen.ts` au prochain `dev` ou `build` (à commiter).
2. Ajoutez l'entrée dans `src/components/app-sidebar.tsx` (tableau `main` ou `ops`), avec le `module` de permission qui conditionne l'affichage.
3. Réutilisez les briques de `src/components/data` (`Toolbar`, `SearchField`, `StatusBadge`, `DataPagination`, `EmptyState`, `ErrorState`) pour rester cohérent avec les autres listes.

8.2 Lire et écrire des données

```
const { data = [], isLoading, error } = useQuery({
  queryKey: ["orders"],
  queryFn: async () => {
    const { data, error } = await supabase.from("orders").select("*, customers(name)");
    if (error) throw error;
    return data;
  },
});

const save = useMutation({
  mutationFn: async (payload) => {
    const { error } = await supabase.rpc("create_order", payload);
    if (error) throw error;
  },
  onSuccess: () => {
    toast.success("Commande créée");
    qc.invalidateQueries({ queryKey: ["orders"] }); // + toutes les listes impactées (stock, ventes...)
  },
  onError: (e: Error) => toast.error(e.message),
});
```

Règles :

- Toute opération qui touche **plusieurs tables ou le stock** doit être une **RPC SQL** transactionnelle, pas une suite d'appels depuis le navigateur.

- Après une action sur le stock, invalidez aussi `["products"]`, `["movements"]` et `["inventory"]`.
- Les messages d'erreur SQL (`RAISE EXCEPTION '...'`) sont affichés tels quels à l'utilisateur : rédigez-les en français clair.
- Confirmez les actions sensibles avec `useConfirm()` et une description qui **dit l'effet sur le stock et l'argent**.

8.3 Ajouter une permission ou un module

1. Migration : `insert into public.permissions(module, action, label) values ('livraisons','view','Voir les livraisons') on conflict do nothing;`, puis accordez-la aux rôles concernés dans `role_permissions`.
2. Politiques RLS de la table avec `public.user_has_permission(auth.uid(), 'livraisons', 'view')`.
3. Front : `const { can } = usePermissions(); can("livraisons", "create")` pour masquer les boutons, et `module: "livraisons"` dans le menu.
4. Vérifiez que l'écran *Rôles & permissions* affiche le nouveau module (les libellés d'actions viennent de `src/routes/_app.roles.tsx`).

8.4 Écrire une fonction SQL (RPC)

- `SECURITY INVOKER` par défaut (respecte RLS); `SECURITY DEFINER` uniquement pour les triggers internes, avec `SET search_path TO 'public'`.
- Idempotence pour les transitions d'état (voir `validate_order`, qui renvoie `already: true`).
- Verrouillez les lignes modifiées (`SELECT ... FOR UPDATE`).
- Renvoyez du `jsonb` avec les numéros créés, pour les messages de l'interface.
- Terminez la migration par rien de spécial : `scripts/migrate.sh` envoie le `NOTIFY pgrst`.

8.5 Modifier un PDF

Les mises en page sont dans `src/lib/quote-pdf.ts`, `order-pdf.ts` et `invoice-pdf.ts` (jsPDF + autotable). L'en-tête des factures utilise `nado-logo-pdf.ts` (« Nado Cuisine », mentions légales `NADO_LEGAL_LINES`). Les autres documents utilisent `nolte-logo-pdf.ts`. `pdf-safe.ts` nettoie les caractères non supportés par les polices PDF standard. Testez en téléchargeant un document réel dans le navigateur.

8.6 Réactiver l'arabe dans l'interface

`src/lib/i18n.tsx` contient les dictionnaires `fr` et `ar`, mais le fournisseur force `lang: "fr"`. Pour réactiver :

1. exposer la vraie valeur `lang` et un sélecteur (Paramètres);
2. poser `dir="rtl"` et `lang="ar"` sur `<html>`;

3. compléter les nombreux libellés codés en dur en français dans les pages (la plupart ne passent pas par `t()`).

Le menu gère déjà le côté droit (`side={lang === "ar" ? "right" : "left"}`), et les classes Tailwind logiques (`ms-` , `me-` , `start-`) sont utilisées.

8.7 Server function avec droits élevés

Modèle : `src/lib/users.functions.ts` .

```
export const maFonction = createServerFn({ method: "POST" })
  .middleware([requireSupabaseAuth]) // JWT de l'appelant → context.userId,
  context.supabase
  .handler(async ({ context, data }) => {
    await assertAdmin(context.supabase, context.userId); // TOUJOURS vérifier avant
    const { supabaseAdmin } = await import("@integrations/supabase/client.server");
    // supabaseAdmin contourne RLS : ne jamais l'exposer au navigateur
  });
```

8.8 Mettre à jour la documentation

Les guides sont en Markdown dans `docs/` . Après modification :

```
docs/outils/generer-pdf.sh
```

Le script construit une image Docker jetable (WeasyPrint et polices Noto, arabe compris) et régénère les trois PDF. Conventions d'encadrés : voir l'en-tête de `docs/outils/md2pdf.py` .

9. Mettre à niveau

Composant	Où	Conseils
Dépendances JS	<code>package.json</code> / <code>bun.lock</code>	<code>bun update</code> par petits lots ; rebuild ; parcours complet de l'app. TanStack Start évolue vite (nitro en bêta 3.0.x) : lire les notes de version avant un saut majeur.
Node	<code>frontend/Dockerfile</code> (<code>node:22-alpine</code>)	Rester ≥ 22 (WebSocket natif requis).
Images Supabase	<code>docker-compose.yml</code> (tags figés)	Mettre à jour une image à la fois, sauvegarde et <code>verify-backup.sh</code> d'abord. GoTrue et Storage appliquent leurs propres migrations au démarrage.
Postgres majeur (15 → 16+)	<code>db</code>	Jamais par simple changement de tag : dump, nouveau volume, restauration. À tester sur une copie.
nginx, certbot, Kong	<code>docker-compose.prod.yml</code> / <code>docker-compose.yml</code>	Changement de tag puis <code>up -d</code> du service ; <code>nginx -t</code> avant.
Supervision	<code>docker-compose.monitoring.yml</code>	Garder les <code>mem_limit</code> . Loki/Promtail : ajouter un swap avant.
OS	Ubuntu	<code>unattended-upgrades</code> gère la sécurité ; redémarrages noyau planifiés hors heures ouvrées (<code>live-restore</code> Docker activé).

Capacité : 3,8 Go de RAM, environ 1 Go libre, **pas de swap**. Un `build app` pendant les heures d'activité peut provoquer un arrêt par manque de mémoire (OOM). Ajouter un fichier d'échange de 2 Go est recommandé avant toute nouvelle charge.

10. Dette technique et problèmes connus

Classés par priorité. **P1** = risque de perte de données ou de sécurité ; **P2** = bug fonctionnel ou incohérence ; **P3** = amélioration.

#	Prio	Problème	Piste de correction
1	P1	Aucune sauvegarde hors site : les 14 archives sont sur le même disque.	Configurer restic (B2 ou S3) → <code>backup-offsite.sh</code> (7.6).
2	P1	Mouvements de stock antidatables : le champ « Date du mouvement » écrit <code>created_at</code> (qui sert aussi d'horodatage d'audit) sans limite ; seul l'admin peut corriger après coup.	Colonne <code>movement_date</code> séparée, <code>created_at = now()</code> ; ou contrainte <code>CHECK</code> / trigger bornant la date.
3	P1	returns / return_items : RLS USING (true) : tout compte connecté peut créer, modifier et supprimer des retours, donc déplacer du stock.	Politiques basées sur une permission (<code>stock.create</code> ou nouveau module <code>returns</code>).
4	P1	Pas de swap , RAM tendue ; plusieurs services sans limite mémoire (db, kong, auth, rest, storage, minio, nginx).	Swap de 2 Go ; limites mémoire.
5	P2	E-mails non fonctionnels (SMTP d'exemple) : alertes, réinitialisation de mot de passe.	<code>scripts/set-gmail-smtp.sh</code> (7.7).
6	P2	Facturation absente du menu (<code>/invoices</code> accessible par URL uniquement).	Ajouter l'entrée dans <code>app-sidebar.tsx</code> (module <code>sales</code>).
7	P2	Double sortie de stock possible : facture <code>pending</code> / <code>paid</code> avec lignes produits d'une commande déjà livrée.	Lier facture ↔ commande/vente et ne pas mouvementer si déjà livré ; ou factures sans effet stock.
8	P2	Interface ≠matrice de permissions : produits, fournisseurs et dépôts sont masqués par <code>isAdmin</code> , alors que la matrice accorde <code>create</code> / <code>update</code> au Manager ; <code>products</code> et <code>warehouses</code> ont une RLS admin-only. Les boutons de commande et de vente s'affichent pour tous, même sans droit (l'action échoue alors avec une erreur RLS).	Utiliser partout <code>can(module, action)</code> et aligner la RLS de <code>products</code> sur <code>user_has_permission</code> .
9	P2	Permissions sans effet : <code>orders.validate</code> , <code>orders.cancel</code> , <code>stock.in/out/adjust</code> et <code>sales.invoice</code> ne sont vérifiées nulle part ; valider exige en réalité <code>orders.update</code> + <code>sales.create</code> . Le Magasinier (qui a <code>validate</code>) ne peut donc pas valider ; le Manager (qui a <code>in/out/adjust</code> mais pas <code>create</code>) ne peut pas saisir de mouvement.	Soit utiliser ces actions dans la RLS et les RPC, soit les retirer du catalogue.
10	P2	Menus mal reliés : <i>Devis</i> s'affiche selon <code>sales.view</code> (RLS sur <code>quotes.view</code>) ; <i>Achats</i> selon <code>suppliers.view</code> (RLS sur <code>purchase_orders.view</code>).	Utiliser le bon module dans <code>app-sidebar.tsx</code> .

#	Prio	Problème	Piste de correction
11	P2	Préférences de notifications décoratives : enregistrées dans <code>localStorage</code> , jamais lues par <code>app-header.tsx</code> .	Les lire dans <code>useNotifications</code> , ou les stocker dans <code>profiles</code> .
12	P2	Clients en double : chaque saisie de nom dans un devis, une commande ou une vente crée une nouvelle ligne <code>customers</code> .	Sélecteur de client existant avec recherche, et création explicite.
13	P3	Arabe désactivé dans l'interface (<code>lang</code> forcé à <code>fr</code> , libellés codés en dur).	Voir 8.6.
14	P3	Libellé métier codé en dur « Prix de Mr Bouhlalla » sur les factures.	Paramétrer ou supprimer.
15	P3	Pas de tests automatisés , pas de CI de lint ou de build avant déploiement.	Au minimum <code>bun run lint + vite build</code> dans le workflow avant le SSH ; tests SQL des RPC de stock.
16	P3	Documentation d'origine obsolète par endroits (<code>DEPLOY.md</code> : Hetzner, domaines génériques ; <code>monitoring/README.md</code> : ports).	Aligner sur ce guide.
17	P3	Liste des mouvements limitée aux 2 000 plus récents dans l'écran Stock ; les mouvements antédats peuvent en sortir.	Pagination serveur.
18	P3	Loki/Promtail prêts mais non démarrés (mémoire).	Après ajout de swap : <code>--profile logs</code> .

11. Checklist de passation

11.1 Accès à transmettre (hors de ce document, par canal sûr)

Accès	Détenteur actuel / emplacement
Dépôt GitHub <code>Ahmed61911/noltekuchen</code> (droits admin)	Propriétaire du compte GitHub
Secrets GitHub Actions (<code>SSH_*</code> , <code>DEPLOY_PATH</code>)	GitHub → Settings → Secrets
Compte OVH (VPS, facturation, console de secours)	Propriétaire du compte OVH
Registrar et DNS de <code>noltenador.digital</code>	À identifier
SSH au serveur (clé publique à ajouter dans <code>~/.ssh/authorized_keys</code>)	Administrateur système
Fichier <code>/opt/nolte/.env</code> (tous les secrets : Postgres, JWT, MinIO, Grafana, exporter, SMTP)	Serveur uniquement, ne jamais le commiter
Mot de passe <code>ops</code> (Grafana/Prometheus)	Seul le hachage est dans <code>nginx/conf.d/ops.htpasswd</code> : le redéfinir si inconnu
Compte administrateur de l'ERP	Utilisateurs admin existants
Boîte <code>noltenador@gmail.com</code> (alertes, mot de passe d'application SMTP)	Propriétaire de la boîte
E-mail Let's Encrypt (<code>LETSENCRYPT_EMAIL</code>)	<code>.env</code>

11.2 Première semaine du nouveau développeur

1. Lire les chapitres 1, 2, 4 et 5 de ce guide.
2. Se connecter en SSH, lancer `$C ps`, ouvrir Grafana, lancer `./scripts/verify-backup.sh`.
3. Monter la pile en local (6.2), restaurer une sauvegarde anonymisée ou de test (6.4).
4. Faire un aller-retour complet : petite modification → branche → PR → merge `main` → vérifier le déploiement (7.3), ou déployer manuellement (7.2).
5. Traiter les points **P1** du chapitre 10, en commençant par la sauvegarde hors site.
6. Activer les e-mails (7.7) et tester une alerte.
7. Révoquer ce qui n'est plus nécessaire : entrée sudoers `90-claude-ufw`, clé de déploiement GitHub si inutile, anciens comptes.

11.3 Contrôles réguliers

Fréquence	Contrôle
Quotidien (automatique)	Sauvegarde 03:00 → <code>logs/nolte-backup.log</code>
Hebdomadaire (automatique)	Restauration test → <code>logs/nolte-verify-backup.log</code> (doit finir par <code>PASS</code>)
Hebdomadaire	Coup d'œil Grafana (disque, mémoire, cibles), boîte d'alertes
Mensuel	Mises à jour d'images mineures, espace disque (<code>docker system df</code>), revue des comptes utilisateurs
Trimestriel	Test de reprise après sinistre sur un VPS jetable (7.10)

Annexe A. Variables d'environnement (`.env`)

Variable	Rôle
<code>NODE_ENV</code> , <code>APP_PORT</code> , <code>APP_URL</code> , <code>SESSION_SECRET</code>	Applicat
<code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_PUBLISHABLE_KEY</code> , <code>VITE_SUPABASE_PROJECT_ID</code>	Injectée build da code na (URL pu l'API, clé
<code>SUPABASE_URL</code>	URL inte Kong po serveur (<code>http://</code> <code>kong:80</code>
<code>SUPABASE_PUBLISHABLE_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>SUPABASE_JWT_SECRET</code>	Clés JWT service_ secret d signatur
<code>POSTGRES_HOST</code> , <code>POSTGRES_PORT</code> , <code>POSTGRES_DB</code> , <code>POSTGRES_USER</code> , <code>POSTGRES_PASSWORD</code>	Base de (superu
<code>POSTGRES_EXPORTER_PASSWORD</code>	Rôle lec seule de supervis
<code>GOTRUE_SITE_URL</code> , <code>GOTRUE_URI_ALLOW_LIST</code> , <code>GOTRUE_DISABLE_SIGNUP</code> , <code>GOTRUE_MAILER_AUTOCONFIRM</code> , <code>GOTRUE_JWT_EXP</code>	Authent
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code> , <code>SMTP_ADMIN_EMAIL</code> , <code>SMTP_SENDER_NAME</code>	E-mails et alerte Grafana
<code>MINIO_ROOT_USER</code> , <code>MINIO_ROOT_PASSWORD</code> , <code>STORAGE_S3_BUCKET</code> , <code>STORAGE_S3_ENDPOINT</code> , <code>STORAGE_S3_REGION</code>	Stockag
<code>DOMAIN_APP</code> , <code>DOMAIN_SUPABASE</code> , <code>LETSENCRYPT_EMAIL</code>	Domain
<code>GRAFANA_ADMIN_PASSWORD</code>	Compte Grafana
<code>RESTIC_REPOSITORY</code> , <code>RESTIC_PASSWORD</code> (+ identifiants fournisseur)	Sauvega site (à c
<code>LOVABLE_*</code>	Migratio unique historiq vider

Annexe B. Commandes utiles

```
# État, logs, ressources
$C ps
$C logs -f --tail=200 app auth rest
docker stats --no-stream
df -h / && docker system df

# Base de données
docker exec -it nolte-db-1 psql -U postgres -d postgres
docker exec nolte-db-1 psql -U postgres -d postgres -c "select * from _schema_migrations order
by 1 desc limit 5"
docker exec nolte-db-1 psql -U postgres -d postgres -c "NOTIFY pgrst, 'reload schema'"

# Stock d'un produit : historique complet
docker exec nolte-db-1 psql -U postgres -d postgres -c \
  "select created_at, type, quantity, stock_before, stock_after, reason from stock_movements
  where product_id = '<uuid>' order by created_at"

# Donner le rôle admin à un utilisateur existant
docker exec nolte-db-1 psql -U postgres -d postgres -c \
  "insert into user_roles(user_id, role) select id, 'admin' from auth.users where email='x@y.z'
  on conflict do nothing"

# nginx
docker exec nolte-nginx-1 nginx -t && docker exec nolte-nginx-1 nginx -s reload

# Sauvegardes
ls -lh backups/ && tail -20 logs/nolte-backup.log
./scripts/backup.sh && ./scripts/verify-backup.sh

# Pare-feu et sécurité
sudo ufw status verbose
sudo fail2ban-client status
```

Document établi le 14/09/2026 à partir du code (commit `ddbe4a2`), de la base et de la configuration de production. En cas de divergence, le code et la base font foi : mettez ce guide à jour (8.8).